Numerical Analysis — Lecture 09

Least Squares Approximation and Polynomial Fitting

by Imelda Trejo Lorenzo – April 06, 2026

Running example: $x_i = 1, \ldots, 10$; $y_i = 1.3, 3.5, 4.2, 5.0, 7.0, 8.8, 10.2, 12.5, 13.0, 15.6$

---

1 The Least Squares Principle

Let $(x_i, y_i)_{i=1}^n$ be n observations and let $f(x; \mathbf{a})$ be a model parameterised by $\mathbf{a} = (a_0, \ldots, a_k)^T$.

Definition 1.1. The *least squares problem* is

$$\min_{\mathbf{a} \in \mathbb{R}^{k+1}} S(\mathbf{a}), \qquad S(\mathbf{a}) \;=\; \sum_{i=1}^{n} \big[y_i - f(x_i; \mathbf{a})\big]^2.$$

A necessary condition for a minimum is that all partial derivatives vanish:

$$\frac{\partial S}{\partial a_j} = 0, \qquad j = 0, 1, \ldots, k.$$

This system of $k+1$ equations in $k+1$ unknowns is called the *normal equations*. When f is linear in $\mathbf{a}$, the normal equations reduce to the linear system $\mathbf{A}^T\mathbf{A}\,\mathbf{c} = \mathbf{A}^T\mathbf{y}$ (Sections 2–3), which has a unique solution whenever $\mathbf{A}$ has full column rank [1] and, as shown in the previous homework.

2 Linear Model and Normal Equations

2.1 From overdetermined system to matrix form

Consider the model $\hat{y} = a_0 + a_1 x$. Each observation yields one equation, producing the overdetermined system $\mathbf{A}\mathbf{c} = \mathbf{y}$ with

$$\mathbf{A} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix} \in \mathbb{R}^{n \times 2}, \qquad \mathbf{c} = \begin{bmatrix} a_0 \\ a_1 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix}.$$

Because $n > 2$ the system is generally inconsistent; instead we seek the *least squares solution*, i.e. the vector $\mathbf{c}^*$ minimising $\|\mathbf{y} - \mathbf{A}\mathbf{c}\|_2^2$.

2.2 Derivation of the normal equations

$$S(a_0, a_1) = \sum_{i=1}^{n} (y_i - a_0 - a_1 x_i)^2.$$

Setting $\partial S/\partial a_0 = 0$:

$$-2\sum_{i=1}^{n}(y_i - a_0 - a_1 x_i) = 0 \quad \Longrightarrow \quad n\,a_0 + \Big(\sum_{i=1}^{n} x_i\Big)a_1 = \sum_{i=1}^{n} y_i.$$

Setting $\partial S/\partial a_1 = 0$:

$$-2\sum_{i=1}^{n} x_i(y_i - a_0 - a_1 x_i) = 0 \quad \Longrightarrow \quad \Big(\sum_{i=1}^{n} x_i\Big)a_0 + \Big(\sum_{i=1}^{n} x_i^2\Big)a_1 = \sum_{i=1}^{n} x_i y_i.$$

In matrix form these are precisely $\mathbf{A}^T\mathbf{A}\,\mathbf{c} = \mathbf{A}^T\mathbf{y}$:

$$\begin{bmatrix} n & \sum x_i \\ \sum x_i & \sum x_i^2 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} \sum y_i \\ \sum x_i y_i \end{bmatrix}.$$

Note: using calculus techniques we arrived at the same normal equations derived from the linear algebra proof obtained in the previous class, namely the projection of $\mathbf{y}$ onto the column space of $\mathbf{A}$.

2.3 How well does the fit explain the data? The coefficient R^2

Once the normal equations are solved and the fitted values $\hat{y}_i$ are in hand, it is natural to ask: *how much of the variability in the data does the model account for?* Define two sums of squares built entirely from quantities already on the board:

$$SS_{\text{res}} \;=\; \sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \;=\; \|\mathbf{y} - \hat{\mathbf{y}}\|^2 \;=\; \|\mathbf{r}\|^2,$$

the squared norm of the residual vector (the value of S at the solution), and

$$SS_{\text{tot}} \;=\; \sum_{i=1}^{n}(y_i - \bar{y})^2 \;=\; \|\mathbf{y} - \bar{y}\mathbf{1}\|^2, \qquad \bar{y} = \tfrac{1}{n}\sum_{i=1}^{n} y_i,$$

the total spread of the data before any model is fitted. The *coefficient of determination* is

$$\boxed{R^2 \;=\; 1 - \frac{SS_{\text{res}}}{SS_{\text{tot}}}}.$$

Geometric reading. Recall that $\hat{\mathbf{y}}$ is the orthogonal projection of $\mathbf{y}$ onto $\text{col}(\mathbf{A})$, so $\mathbf{r} \perp \hat{\mathbf{y}}$. When the model includes an intercept, the Pythagorean theorem applied to this decomposition gives $SS_{\text{tot}} = SS_{\text{reg}} + SS_{\text{res}}$, where $SS_{\text{reg}} = \|\hat{\mathbf{y}} - \bar{y}\mathbf{1}\|^2$. Thus $R^2 = SS_{\text{reg}}/SS_{\text{tot}}$ is the *fraction of the total variation in* $\mathbf{y}$ *captured by the column space of* $\mathbf{A}$ — equivalently, the square of the cosine of the angle between $\mathbf{y} - \bar{y}\mathbf{1}$ and its projection. No probability or statistical inference is required; this is a statement purely about the geometry of least squares.

Range and meaning. $0 \le R^2 \le 1$; a value near 1 means $\mathbf{r}$ is small relative to the data spread, while $R^2 = 0$ means the fitted model does no better than the constant predictor $\hat{y} \equiv \bar{y}$.

A word of caution for researchers. R^2 measures *in-sample fit only.* It says nothing about whether the model is correct, whether it generalises to new data, or whether a high value is scientifically meaningful. We will return to honest model evaluation — penalised criteria, cross-validation, residual diagnostics — later in the course. For now, cultivate one habit: **always inspect the residual vector r, not just its norm.**

2.4 Numerical example

The required sums are $n = 10$, $\sum x_i = 55$, $\sum x_i^2 = 385$, $\sum y_i = 81.1$, $\sum x_i y_i = 520.9$, giving

$$\begin{bmatrix} 10 & 55 \\ 55 & 385 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \end{bmatrix} = \begin{bmatrix} 81.1 \\ 520.9 \end{bmatrix} \quad \Longrightarrow \quad a_0 = -0.3600, \quad a_1 = 1.5382,$$

so $\hat{y} = -0.36 + 1.5382\,x$, with $R^2 = 0.9957$.

3 Polynomial Fitting — The General Case

For the degree-k polynomial $\hat{y} = a_0 + a_1 x + \cdots + a_k x^k$ the design matrix is the Vandermonde matrix

$$\mathbf{A} = \begin{bmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^k \\ 1 & x_2 & x_2^2 & \cdots & x_2^k \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \cdots & x_n^k \end{bmatrix} \in \mathbb{R}^{n \times (k+1)},$$

and the normal equations $\mathbf{A}^T \mathbf{A}\,\mathbf{c} = \mathbf{A}^T \mathbf{y}$ take the explicit form

$$\begin{bmatrix} n & \sum x & \sum x^2 & \cdots & \sum x^k \\ \sum x & \sum x^2 & \sum x^3 & \cdots & \sum x^{k+1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \sum x^k & \sum x^{k+1} & \cdots & \cdots & \sum x^{2k} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_k \end{bmatrix} = \begin{bmatrix} \sum y \\ \sum xy \\ \vdots \\ \sum x^k y \end{bmatrix}.$$

The (i, j) entry of $\mathbf{A}^T \mathbf{A}$ equals $\sum_{i=1}^{n} x_\ell^{i+j-2}$; the matrix is symmetric positive semi-definite and positive definite whenever the x_i are not all equal [**?**].

Example 3.1 (Quadratic fit to the running data)**.** For $k = 2$: $\sum x^3 = 3025$, $\sum x^4 = 25333$, $\sum x^2 y = 4537.7$,

$$\begin{bmatrix} 10 & 55 & 385 \\ 55 & 385 & 3025 \\ 385 & 3025 & 25333 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} 81.1 \\ 520.9 \\ 4537.7 \end{bmatrix},$$

yielding $a_0 = 0.3817$, $a_1 = 1.1692$, $a_2 = 0.0337$, $R^2 = 0.9915$. Note that R^2 decreased relative to the linear fit (0.9957) because the quadratic term is not warranted for these data.

4 Linearisable Nonlinear Models

When the model is not polynomial, a variable substitution can reduce it to a linear form; least squares then operates on the transformed variables.

Exponential model $y = \alpha\, e^{\beta x}$. Taking logarithms: $\ln y = \ln \alpha + \beta x$. Setting $Y_i = \ln y_i$ and $A = \ln \alpha$ gives the linear model $Y = A + \beta x$; apply least squares to (x_i, Y_i) and recover $\alpha = e^A$.

Power model $y = \alpha\, x^{\beta}$. Taking logarithms: $\ln y = \ln \alpha + \beta \ln x$. Setting $Y_i = \ln y_i$ and $X_i = \ln x_i$ gives $Y = A + \beta X$; apply least squares to (X_i, Y_i) (log–log space).

Remark 4.1. Linearisation minimises $\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$, not $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$. The two criteria are inequivalent because the transformation changes the error structure. When intrinsic nonlinear fitting is required, use the Gauss–Newton or Levenberg–Marquardt algorithms [**?**, **?**].

5 Extension to Multiple Regression and GLMs

Multiple linear regression. With p predictors write $\mathbf{A} \in \mathbb{R}^{n\times p}$ and $\boldsymbol{\beta} \in \mathbb{R}^p$. The least squares estimator is

$$\hat{\boldsymbol{\beta}} = (\mathbf{A}^T\mathbf{A})^{-1}\mathbf{A}^T\mathbf{y},$$

provided $\mathbf{A}$ has rank p. When $\mathbf{A}^T\mathbf{A}$ is ill-conditioned, ridge regression adds a penalty $\lambda\|\boldsymbol{\beta}\|^2$, replacing the normal equations by $(\mathbf{A}^T\mathbf{A} + \lambda I)\boldsymbol{\beta} = \mathbf{A}^T\mathbf{y}$ [**?**].

Generalised Linear Models (GLM). A GLM specifies a link function g and an exponential-family distribution:

$$g(\mu_i) = \mathbf{x}_i^T\boldsymbol{\beta}, \qquad \mu_i = \mathbb{E}[y_i \mid \mathbf{x}_i].$$

The log-likelihood $\ell(\boldsymbol{\beta}) = \sum_{i=1}^{n} \log f(y_i; \mu_i)$ replaces the sum of squares as the objective. For the Gaussian family, maximising ℓ is exactly ordinary least squares. For other families (Poisson, Binomial), the maximum likelihood equations are solved by *Iteratively Reweighted Least Squares* (IRLS): at each iteration the weighted normal equation

$$(\mathbf{A}^T\mathbf{W}^{(t)}\mathbf{A})\,\boldsymbol{\beta} \;=\; \mathbf{A}^T\mathbf{W}^{(t)}\mathbf{z}^{(t)}$$

is solved, where $\mathbf{W}^{(t)}$ is a diagonal weight matrix and $\mathbf{z}^{(t)}$ is an adjusted dependent variable [**?**, **?**].

Gradient descent. When p is too large to invert $\mathbf{A}^T\mathbf{A}$ (e.g. deep neural networks), one iterates

$$\boldsymbol{\beta}^{(t+1)} = \boldsymbol{\beta}^{(t)} - \eta\, \nabla_{\boldsymbol{\beta}} L(\boldsymbol{\beta}^{(t)}), \qquad L = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2.$$

Stochastic variants (SGD, Adam) approximate the gradient on mini-batches. The objective L is still the mean squared error — the same functional as in Section 1 [**?**].

6 Laboratory: Linear Regression in R

The following script, written by I. Trejo (2025), illustrates how the theoretical framework of Sections 1–6 is implemented in practice using the `lm` function in R. The dataset `cdc_diet.xls` contains measurements of cost, weight (`wt_oz`), height (`ht_cm`), and diet group for a sample of subjects.

```
#
    ##############################################################
# Objective: implementation of linear regression models
# Last update: March 18th, 2025
# By Imelda Trejo
#
    ##############################################################


library(readxl)
library(ggplot2)
library(viridis)   # perceptually uniform colour scales

rm(list = ls())    # clear workspace

myPath <- paste0("C:/Users/Imelda/OneDrive - UNIVERSIDAD
    NACIONAL ",
                 "AUTONOMA DE MEXICO/CCM_UNAM/Teaching/",
                 "Numerical_analysis/lab_2025/data/")
data <- read_excel(paste0(myPath, "cdc_diet.xls"))
data[, 4] <- as.factor(data[, 4])   # diet as categorical
    variable

# ---- Exploratory plots
    ------------------------------------------------
ggplot(data, aes(x = cost,  y = wt_oz, group = diet, color =
    diet)) +
  geom_point(size = 2)

ggplot(data, aes(x = cost,  y = ht_cm, group = diet, color =
    diet)) +
  geom_point(size = 2)

ggplot(data, aes(x = ht_cm, y = wt_oz, group = diet, color =
    diet)) +
  geom_point(size = 2)

# ---- Example 1: simple linear regression  wt_oz ~ ht_cm
    -------------
fit_altura_peso <- lm(wt_oz ~ ht_cm, data = data)
summary(fit_altura_peso)

ggplot(data, aes(x = ht_cm, y = wt_oz)) +
```

```
  geom_point(size = 2) +
  geom_smooth(method = "lm")

# ---- Example 2: multiple regression  cost ~ ht_cm + diet ------------
fit_c <- lm(cost ~ ht_cm + diet, data = data)
summary(fit_c)

fit_c$coefficients[3]   # coefficient for the second diet level
```

Listing 1: Linear regression in R (I. Trejo, 2025).

Connections to theory. `lm` solves the normal equations $\mathbf{A}^T\mathbf{A}\,\hat{\boldsymbol{\beta}} = \mathbf{A}^T\mathbf{y}$ via a QR decomposition of $\mathbf{A}$, which is more numerically stable than inverting $\mathbf{A}^T\mathbf{A}$ directly [**?**]. In Example 1 the design matrix has two columns $(1, x_i)$, exactly as in Section 2. In Example 2, R automatically appends indicator columns for the factor levels of `diet`, illustrating the multiple-regression framework of Section 6. The `summary` output reports R^2, R^2_{adj}, and t-tests for each coefficient.

References

References

[1] Kincaid, D., Cheney, W., *Numerical Analysis of Scientific Computing*. Brooks/-Cole Publishing Company, 1991.